

Robotics Subsystem Documentation

1. Overview of the Robotics Subsystem

The robotics subsystem of the quadruped robot is responsible for enabling autonomous motion, coordination of multiple joints, and execution of locomotion patterns. It serves as the intelligence layer that bridges the mechanical structure with software control, allowing the robot to perform stable and repeatable walking behaviours.

This subsystem integrates the following key components:

- ROS 2 based software architecture
- Inverse kinematics (IK) for leg motion
- Gait generation and synchronization
- Joint-level position control
- Real-time command publishing

The primary objective of the robotics subsystem is to generate smooth, stable, and physically feasible joint trajectories for all four legs, enabling reliable quadruped locomotion.

2. Joint and Kinematic Structure and URDF setup

The quadruped robot consists of **four legs**, each with **three actuated joints**:

1. Hip rotation joint (servo rotor)
2. Thigh joint (upper leg)
3. Knee joint (lower leg)

This results in a total of **12 degrees of freedom (DOF)**. All joints are defined as **revolute joints** with specified motion limits, effort limits, and velocity limits. These limits ensure that the simulated robot operates within physically feasible and mechanically safe ranges.

Each joint also includes an axis definition, which determines the direction of rotation. Due to mirrored mechanical assembly, joint axes differ across legs, which is later compensated in software using joint sign tables.

2.1. Inertial and Collision Modelling

Each link in the URDF includes:

- Mass value
- Center of mass location
- Inertia tensor
- Collision mesh
- Visual mesh

These parameters allow Ignition Gazebo to compute realistic:

- Gravitational effects
- Joint torques
- Contact forces
- Dynamic stability

Accurate inertial modelling is essential for achieving realistic legged locomotion behaviour in simulation.

2.2. ROS 2 Control Integration

To enable real-time control of the robot in simulation, the URDF integrates **ROS 2 Control** using the following block:

```
<ros2_control name="PetBotSystem" type="system">
```

This block defines a hardware interface that exposes each joint to ROS 2 as a controllable actuator. For every joint, the following interfaces are provided:

- Position command interface
- Position state interface

- Velocity state interface

This allows external ROS 2 nodes (such as gait controllers and teleoperation modules) to directly send joint position commands to the simulated robot.

2.3. Transmission Blocks

Each joint is associated with a **transmission block**, which represents the mapping between the software command interface and the physical actuator.

The transmission blocks use:

```
<!-- FRONT LEFT LEG -->
<transmission name="trans_servoroterf1">
  <type>transmission_interface/SimpleTransmission</type>
  <joint name="servoroterf1">
    <hardwareInterface>hardware_interface/PositionCommandInterface</hardwareInterface>
  </joint>
  <actuator name="servoroterf1_motor">
    <hardwareInterface>hardware_interface/PositionCommandInterface</hardwareInterface>
    <mechanicalReduction>1</mechanicalReduction>
  </actuator>
</transmission>
```

These blocks define:

- Which joint is being controlled
- Which actuator drives the joint
- Mechanical reduction ratio

The transmission layer acts as a bridge between:

control algorithms (software level)
and actuators (hardware level)

Without transmission blocks, ROS 2 cannot propagate control commands to the joints, making them essential for robot actuation.

2.4. Ignition Gazebo Simulation Plugin

The robot is simulated using **Ignition Gazebo**, which is a modern, high-performance robotics simulation environment.

The following plugin is used:

```
<plugin filename="libign_ros2_control-system.so"
        name="ign_ros2_control::IgnitionROS2ControlPlugin">
```

This plugin enables **bidirectional communication between Ignition and ROS 2**, allowing:

- ROS 2 nodes to control joints in Ignition
- Ignition to publish joint states back to ROS 2
- Real-time physics simulation with closed-loop control

The plugin loads controller parameters from an external YAML configuration file, which defines position controllers for all joints.

2.5. Controller Configuration Using YAML File

A YAML file is used to set up and manage how the robot joints are controlled. Instead of writing all control settings inside the program, these settings are kept in a separate YAML file, which makes the system easier to understand and modify.

The YAML file tells ROS 2:

- which joints can be controlled,
- what type of controller is used,
- and how commands should be applied to the joints.

In this project, a **forward position controller** is used. This means the robot is controlled by directly sending desired joint angles. These commands are sent to the topic:

`/forward_position_controller/commands`

The YAML file is loaded by the Ignition simulation plugin, so Ignition knows how to apply the commands coming from ROS 2. This makes the robot behave in simulation the same way it would behave in real hardware.

Using a YAML file makes the control system more flexible, because changes in controller settings can be done easily without changing the main control code.

2. Software Architecture Using ROS 2

The entire control framework is implemented using **ROS 2 (Robot Operating System 2)**, which provides a modular and scalable middleware for robot software development.

The robotics subsystem follows a **node-based architecture**, where:

- A dedicated ROS 2 node generates joint commands.
- Commands are published on the topic:
`/forward_position_controller/commands`
- The message type used is:
`std_msgs/Float64MultiArray`

Each message contains **12 joint values**, corresponding to:

- 3 joints per leg (hip, thigh, knee)
- 4 legs: Front Left (FL), Front Right (FR), Rear Left (RL), Rear Right (RR)

This architecture allows real-time communication between the control algorithm and the simulated or physical robot.

3. Predefined Pose Representation

Each posture is defined as a **12-element joint configuration vector**, corresponding to the three joints (hip, thigh, knee) of all four legs. The predefined poses include:

- **Stand pose** – all joints set to neutral configuration.

- **Sit pose** – front and rear legs bend symmetrically to lower the body.
- **Crouch pose** – legs bend further to bring the robot close to the ground.

To ensure mechanical symmetry, mirrored joint values are carefully assigned for left and right legs. This is particularly important for knee joints, where the left side requires sign inversion due to mirrored servo mounting.

This explicit pose definition allows the robot to achieve consistent and physically stable postures without relying on inverse kinematics.

3.1 Smooth Interpolated Motion

To prevent sudden joint movements and mechanical stress, transitions between poses are performed using **linear interpolation**. Instead of directly commanding the target pose, the system gradually interpolates from the current pose to the target pose over a fixed duration.

Mathematically, for each joint:

$$\theta(t) = \theta + \alpha(\theta_{\text{target}} - \theta_{\text{start}})$$

where:

- $\alpha \in [0, 1]$ is the interpolation factor,
- θ_{start} is the current joint angle,
- θ_{target} is the desired joint angle.

This ensures:

- Smooth posture transitions
- Reduced torque spikes
- Improved actuator safety
- Increased simulation stability

Such interpolation-based motion is commonly used in real robotic systems to avoid impulsive control commands.

3.2 Keyboard-Based Teleoperation Interface

The module uses low-level terminal input to capture keyboard commands in real time. The following control keys are implemented:

Key	Action
W	Move to Stand pose
S	Move to Sit pose
C	Move to Crouch pose
Q	Exit control program

This interface allows the user to directly control the robot's posture without requiring external hardware such as joysticks or game controllers.

3.3 Importance in Robotics System

The manual pose control module plays a critical role in the overall robotics subsystem:

- Enables **safe initial testing** of mechanical joints.
- Allows **calibration of joint directions and limits**.
- Provides **human override** during experiments.
- Useful for **demonstrations and presentations**.
- Helps in debugging gait-related instability.

In practical robotics development, such teleoperation tools are essential for system bring-up, hardware validation, and emergency intervention.

4. Inverse Kinematics for Leg Control

To control the position of each leg, an **analytical inverse kinematics (IK)** model was implemented for a 2-degree-of-freedom planar leg.

Each leg is modelled as a two-link manipulator:

- Thigh length: L_1
- Knee (lower leg) length: L_2

Given a desired foot position (x, z) , the IK solver computes:

- Thigh joint angle
- Knee joint angle

using trigonometric relationships derived from the **law of cosines**.

This allows precise control of foot placement in Cartesian space, enabling the generation of biologically inspired walking patterns.

5. Gait Generation and Locomotion Strategy

The robot uses a **diagonal trot gait**, which is one of the most stable and commonly used quadruped walking patterns.

In this gait:

- Front Left (FL) and Rear Right (RR) move together
- Front Right (FR) and Rear Left (RL) move together
- At any time, two legs remain in contact with the ground

This ensures continuous support of the robot's center of mass, resulting in improved stability.

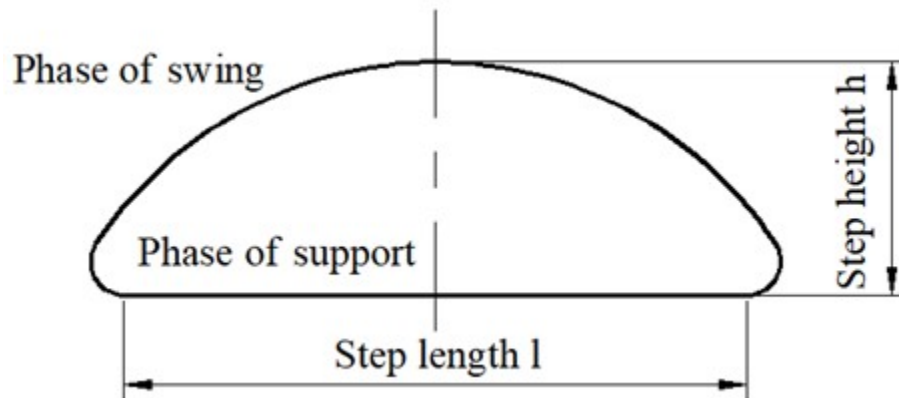
A periodic gait cycle is defined using a normalized phase variable:

$$\text{phase} \in [0, 1]$$

which controls the swing and stance phases of each leg.

6. Foot Trajectory Planning

Foot trajectory planning is a fundamental component of legged locomotion, as it directly determines how each leg interacts with the ground during walking. In the proposed quadruped system, the motion of each foot is divided into two distinct phases: the **swing phase** and the **stance phase**. Each phase is designed using a different mathematical profile to achieve smooth, stable, and physically realistic motion.



The foot trajectory function clearly separates the two phases:

- ❖ **Swing phase**
 - Sinusoidal vertical motion
 - Forward-moving foot
 - ❖ **Stance phase**
 - Constant vertical height
 - Linear backward motion relative to body
-

6.1 Swing Phase (Sinusoidal Trajectory)

The **swing phase** corresponds to the period when a leg is lifted off the ground and moved forward to prepare for the next step. During this phase, the foot must follow a smooth trajectory that avoids sudden accelerations and ensures gentle landing.

In this work, the vertical motion of the foot during the swing phase is generated using a **sinusoidal function**. The sinusoidal profile gradually lifts the foot from the ground, reaches a maximum height at the midpoint of the swing, and then smoothly lowers it back to the ground.

Mathematically, this can be represented as:

$$z(s)=h-H\sin(\pi s)$$

where:

- h is the nominal body height,
- H is the step height,
- $s \in [0, 1]$ is the normalized swing phase variable.

This sinusoidal motion ensures that both the velocity and acceleration of the foot remain continuous, which significantly reduces jerks and impact forces. As a result, the foot does not collide abruptly with the ground, leading to smoother motion and improved mechanical safety.

6.2 Stance Phase (Linear Trajectory)

The **stance phase** occurs when the foot is in contact with the ground and supports the robot's body weight. During this phase, the foot should ideally remain at a constant height while moving backward relative to the body, simulating ground contact as the robot moves forward.

In this system, the stance phase is modelled using a **linear trajectory** along the forward direction (x-axis), while maintaining a constant vertical position. This creates the effect that the body is moving forward over a stationary foot.

Mathematically:

$$x(s)=x_{\max}-s \cdot L$$

where:

- L is the step length,
- $s \in [0, 1]$ is the normalized stance phase variable.

This linear profile provides a constant velocity motion, which ensures uniform propulsion and stable load transfer between legs.

6.3 Combined Effect and Advantages

By combining a **sinusoidal swing trajectory** with a **linear stance trajectory**, the robot achieves a natural and efficient walking pattern.

This approach offers several important benefits:

- Smooth lifting and landing of the foot during swing.
- Stable ground support during stance.
- Continuous and physically realistic joint motion.
- Reduced mechanical stress on joints and actuators.
- Improved simulation stability and real-world reliability.

This method closely resembles biological locomotion observed in animals, where the swing leg follows a curved trajectory while the stance leg provides linear support. Consequently, the adopted trajectory planning strategy provides an optimal balance between simplicity, stability, and realism for quadruped locomotion control.

7. Hybrid Joint Control Strategy

A **hybrid control approach** is adopted in the robotics subsystem:

- Thigh joints follow IK-generated trajectories.
- Knee joints can be:
 - Fully controlled by IK, or
 - Temporarily frozen during debugging and calibration.

This strategy allows:

- Isolation of problematic joints
- Safe testing of individual components
- Progressive activation of full leg dynamics

This method is commonly used in real robotics labs to debug complex multi-joint systems.

8. Joint Sign Table and Motor Direction Mapping

Due to physical constraints in mechanical assembly and servo motor mounting, not all joints in the quadruped robot rotate in the same direction for the same mathematical input. Some joints are mirrored, especially between left and right legs and between front and rear legs.

To handle this, a **joint sign table** is implemented in the robotics subsystem. This table ensures that the inverse kinematics outputs are correctly mapped to the actual motor directions of each joint.

The joint sign table is derived from Rviz simulation. Signs are utmost important for symmetric application of the IK over all the four legs.

8.1 Motivation for Using a Sign Table

In practical robotic systems:

- Servo motors are mounted in mirrored orientations.
- Joint axes in the URDF may be inverted.
- Electrical wiring may differ between joints.

As a result, directly applying IK angles to all joints leads to incorrect or asymmetric motion. The sign table solves this problem by introducing a per-leg directional correction.

8.2 Hip Sign Table

The hip sign table defines the direction in which the hip joint should rotate for each leg:

Leg	Hip Sign
Front Left (FL)	-1
Front Right (FR)	+1
Rear Left (RL)	+1
Rear Right (RR)	-1

This ensures symmetric rightward and leftward motion across all legs despite mirrored joint orientations.

8.3 Thigh Sign Table

The thigh sign table defines the direction in which the thigh joint should rotate for each leg:

Leg	Thigh Sign
Front Left (FL)	+1
Front Right (FR)	-1
Rear Left (RL)	-1
Rear Right (RR)	-1

This ensures symmetric forward and backward motion across all legs despite mirrored joint orientations.

8.4 Knee Sign Table

Similarly, the knee sign table corrects the bending direction of the knee joints:

Leg	Knee Sign
Front Left (FL)	+1
Front Right (FR)	-1
Rear Left (RL)	+1
Rear Right (RR)	-1

This guarantees that all knees bend in a physically consistent manner, even though their motors may be mounted in opposite directions.

8.5 Application in Control Law

The sign tables are applied directly in the joint control equations:

$$\Theta_{\text{hip}} = \theta_{\text{neutral}} + (\text{Hip sign}) * \theta_{\text{IK}}$$

$$\Theta_{\text{thigh}} = \theta_{\text{neutral}} + (\text{Thigh sign}) * \theta_{\text{IK}}$$

$$\Theta_{\text{knee}} = \theta_{\text{neutral}} + (\text{Knee sign}) * \theta_{\text{IK}}$$

Where:

- θ_{IK} is the angle computed from inverse kinematics.
 - θ_{neutral} is the neutral standing angle.
 - The sign tables ensure correct physical motion.
-

8.6 Importance of the Sign Table

The sign table plays a critical role in system stability and correctness:

- Enables a **single IK model** to be used for all legs.
 - Eliminates the need for separate kinematic equations.
 - Compensates for mechanical and URDF asymmetries.
 - Prevents mirrored or inverted leg motion.
 - Simplifies debugging and calibration.
-

9. Startup Safety and Initialization

To prevent sudden dynamic instability at launch, a **startup safety mechanism** is implemented:

- The robot holds the neutral pose for a fixed duration.
- During this period, no gait motion is applied.
- This allows physics and joints to settle before locomotion begins.

Such startup delays are standard practice in real robotic systems to avoid actuator shocks and simulation divergence.

10.Introduction of World-Referenced Stance Motion

In this version, the foot trajectory is modified to approximate **world-frame stance behavior**.

During the **stance phase**, the foot is intended to remain relatively stationary with respect to the ground while the robot body moves forward. This creates the effect of **actual forward locomotion**, instead of feet sliding symmetrically about the body frame.

This approach improves realism compared to purely body-centered trajectories and brings the gait closer to real quadruped walking behavior.

In simple gait controllers, the foot moves **forward and backward symmetrically** with respect to the robot body.

That means:

- The body does **not actually move forward**
- The feet just slide back and forth under the robot
- This looks like “walking in place”

World-frame stance (what is implemented)

In **real walking**:

- During **stance**, the foot stays **fixed on the ground**
- The **body moves forward over the foot**
- During **swing**, the foot moves forward to a new contact point

```
def foot_position(s, swing):
    if swing:
        x = -STEP_LENGTH/2 + STEP_LENGTH * s + 0.04
        z = (BODY_HEIGHT-STEP_HEIGHT* math.sin(math.pi * s))
    else:
        x = STEP_LENGTH/2 - STEP_LENGTH * s -0.02
        z = BODY_HEIGHT
    return x, z
```

What this does:

- $-\text{STEP_LENGTH}/2 + \text{STEP_LENGTH} * s$
→ normal swing from back to front
 - $+ 0.04$
→ **extra forward bias**
-

11. System Calibration and Debugging Modes

Several debugging modes were implemented during development, including:

- Thigh-only motion mode (knees frozen)
- Single diagonal leg motion
- Static stand mode (no gait)

These modes enable:

- Verification of joint directions
- Calibration of sign conventions
- Detection of mechanical asymmetries
- Safe incremental testing

This systematic debugging approach significantly improves system reliability and reduces integration errors.

12. Future Implementation of various other Gaits and Motion

12.1. Backward Motion

At present, the robot is capable of forward walking by biasing the swing foot placement in the forward direction. Backward locomotion can be achieved by **reversing the foot trajectory logic**.

This can be implemented by:

- Placing the swing foot behind the body instead of ahead.
- Increasing backward displacement during the swing phase.
- Reducing or reversing the stance phase retreat.

By altering the sign and magnitude of the forward offsets in the foot trajectory equations, the same inverse kinematics framework can be reused to generate backward walking without additional mechanical changes.

12.2. Sideward (Lateral) Motion

Sideward motion, also known as lateral walking, can be implemented by extending the gait logic to include **lateral foot displacement**.

This can be achieved by:

- Modifying hip joint angles dynamically during the gait cycle.
- Introducing a lateral offset in foot placement for swing legs.
- Adjusting diagonal gait timing to maintain stability during side motion.

Such motion would allow the robot to perform sideways translation, which is useful for obstacle avoidance and alignment tasks.

12.3. Turning and Directional Control

Turning motion can be achieved by commanding **different step lengths for left and right legs**.

Possible approaches include:

- Increasing step length on one side while decreasing it on the other.
- Applying asymmetric hip offsets.
- Modifying swing timing between left and right legs.

This would enable smooth turning without changing the overall gait structure.

13. Scope of Improvement

13.1. Closed-Loop COM Stabilization

The current system relies on open-loop, gait-based stability. Future work can introduce **closed-loop center-of-mass stabilization** by integrating sensor feedback.

Potential enhancements include:

- IMU-based roll and pitch feedback.
- Feedback control of hip and thigh joints.
- Dynamic adjustment of foot placement based on stability errors.

This would significantly improve robustness on uneven terrain.

13.2. Adaptive Gait Parameters

Currently, gait parameters such as step length, step height, and cycle time are fixed. In future implementations, these parameters can be made **adaptive** based on:

- Walking speed commands

- Terrain conditions.
- Stability requirements.

This would allow the robot to switch between slow, stable walking and faster locomotion.

13.3. Sensor Integration and Odometry

Future versions of the controller can integrate:

- Leg odometry.
- Ground contact detection.
- IMU-based state estimation.

These additions would enable accurate estimation of robot motion in the world frame and support more advanced control strategies such as trajectory tracking and navigation.

13.4. Learning-Based Gait Optimization

As an advanced extension, machine learning techniques such as reinforcement learning can be used to optimize gait parameters automatically. This would allow the robot to learn efficient walking patterns through interaction with the environment.

14. Conclusion

The robotics subsystem provides a complete and modular control framework for quadruped locomotion. By integrating ROS 2, inverse kinematics, gait generation, and hybrid joint control, the system achieves:

- Stable and repeatable walking behaviour
- Smooth joint trajectories
- Robust startup and debugging capabilities
- Strong coupling between software intelligence and mechanical structure

The developed robotics subsystem forms a solid foundation for future extensions, including sensor feedback, closed-loop balance control, obstacle avoidance, and learning-based locomotion strategies.

Overall, the subsystem successfully transforms the mechanical quadruped structure into an intelligent, programmable, and autonomous robotic platform.